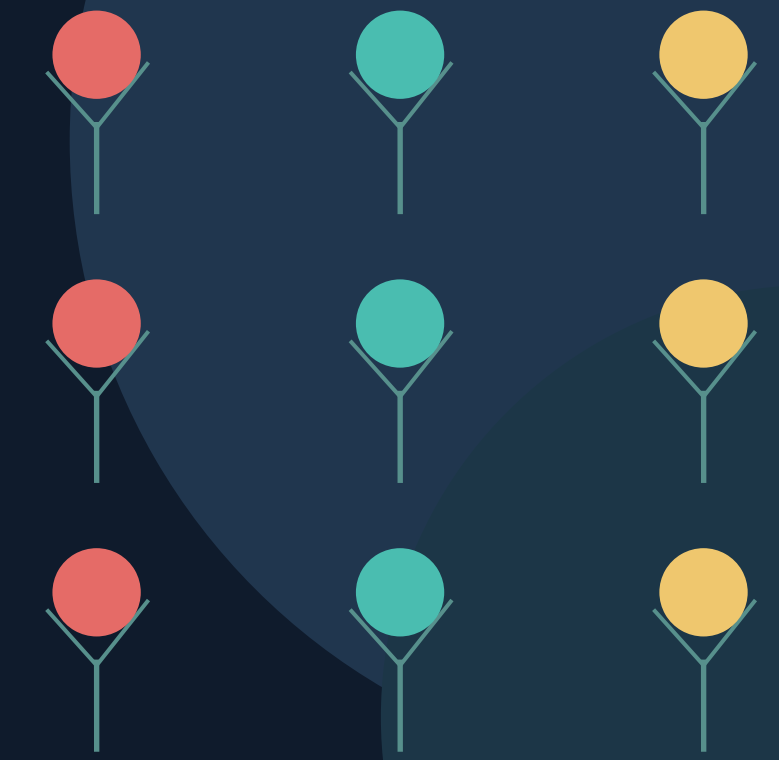
ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ · 2026

СЛУЧАЙНЫЙ ЛЕС

От интуиции до ограничений



Рзаева Элеонора / Медведьков Роман

Команда 17

НАВИГАЦИЯ

План доклада

01 Теория и интуиция

02 Алгоритм и OOB

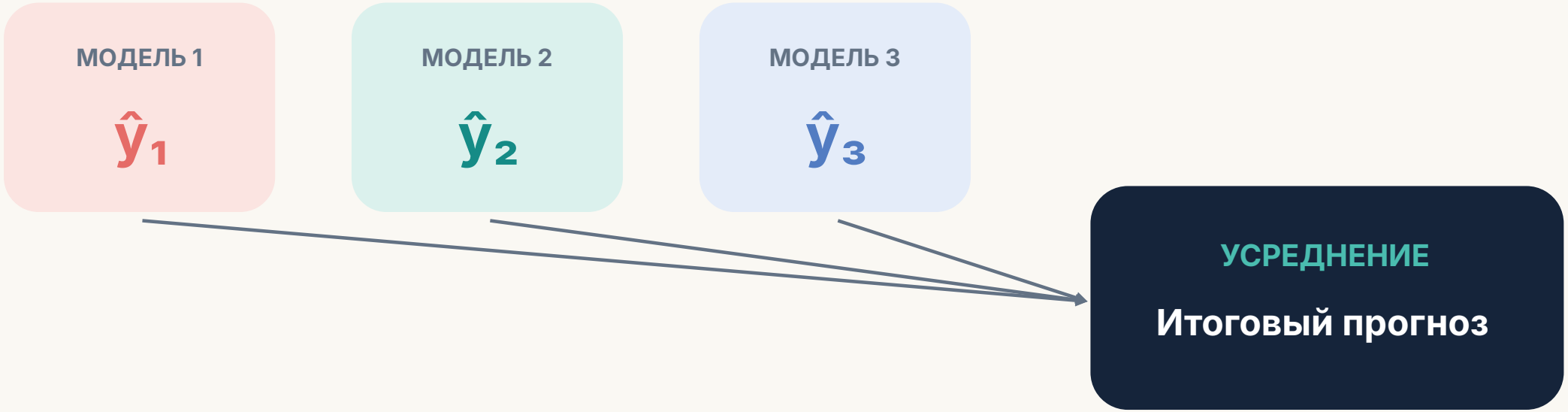
03 Интерпретация и ловушки

04 Сравнение с альтернативами

05 Выводы

Команда 17

Ансамбль: сила коллективного решения



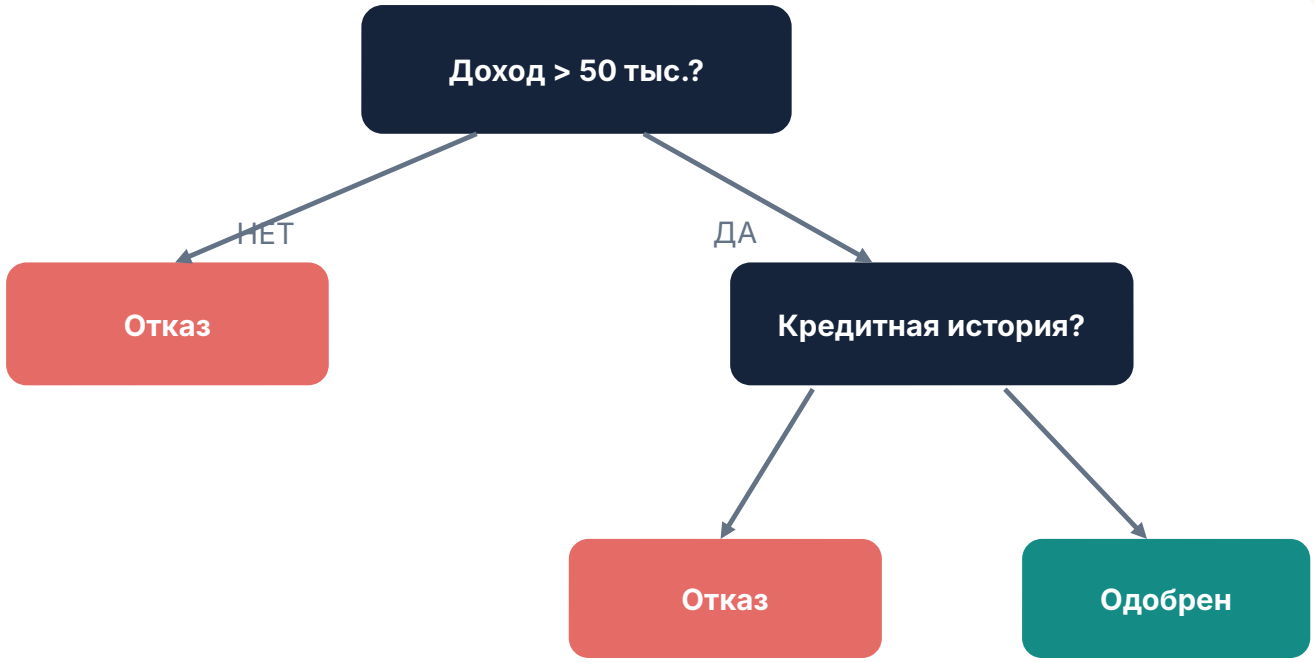
Ансамбль = несколько моделей вместе

Усреднение снижает дисперсию

Ошибка = смещение + дисперсия

Аналогия: решение группы вместо решения одного человека

Решающее дерево: база



Внутренний узел

Условие $x_j \le t$

Сплит

Gini или энтропия

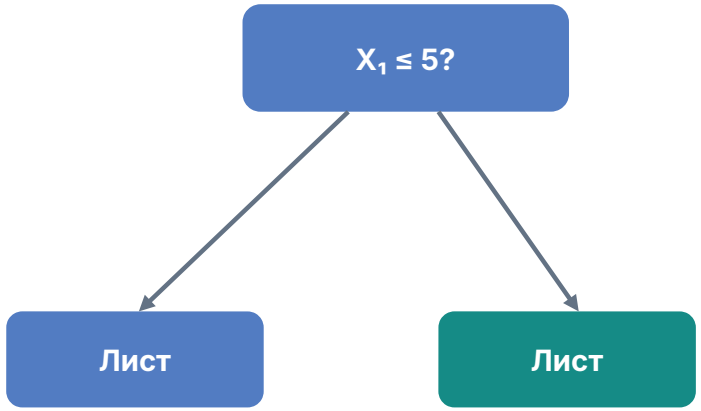
Лист = константа

Мода класса / среднее значение

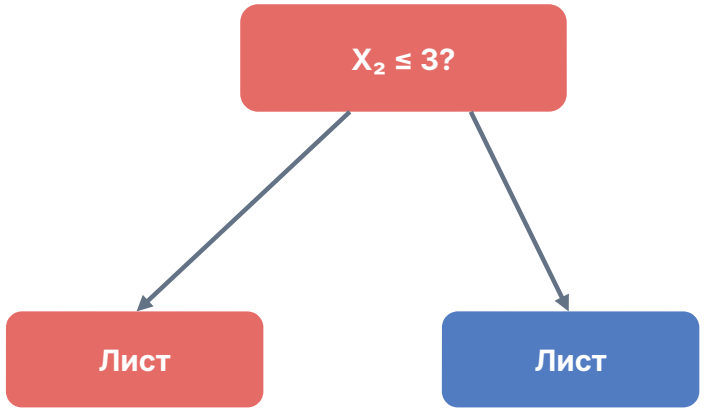
Связь с докладом команды 9: «Метод решающих деревьев»

Проблема: высокая дисперсия

ВЫБОРКА A



ВЫБОРКА B: −1 ОБЪЕКТ

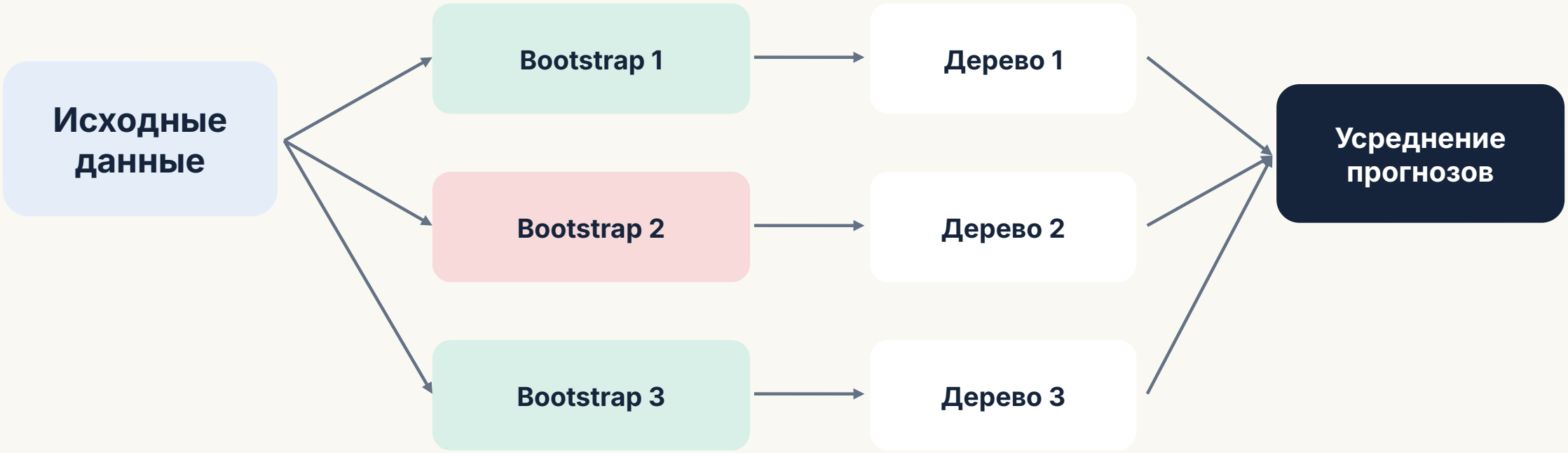


Малое изменение данных → другое дерево

100% Accuracy на обучении

85% Accuracy на тесте

Bagging: Bootstrap Aggregating

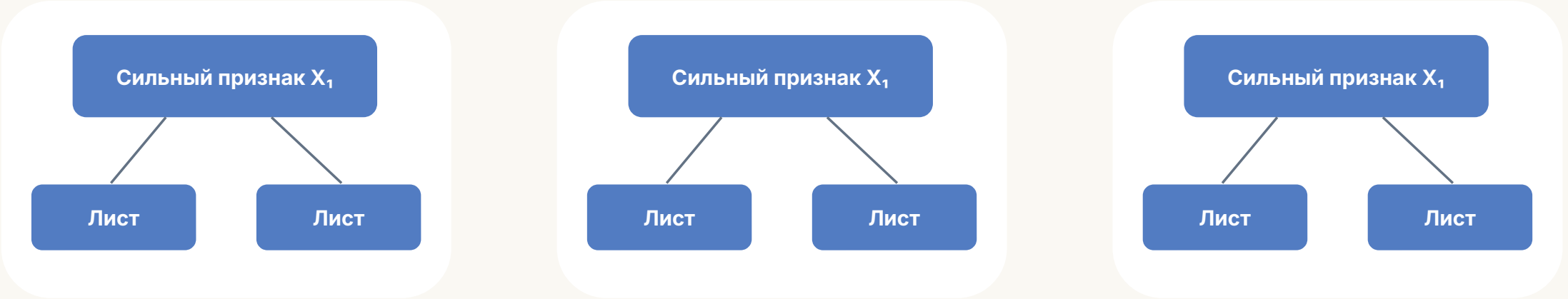


$$\mathrm{Var}(\bar{f}) = \sigma^2 / B$$

Для независимых одинаково распределённых моделей

Bootstrap = выборка с возвращением • Aggregating = усреднение

Проблема: деревья скоррелированы



$$\mathrm{Var}(\bar{f}) = \rho\sigma^2 + ((1-\rho)/B)\sigma^2$$

$\rho\sigma^2$ не исчезает при $B \to \infty$

Нужно уменьшить корреляцию ρ между деревьями

Random Forest: два источника случайности

01

BOOTSTRAP

Случайные объекты
с возвращением

Лео Брейман · 2001

02

MAX_FEATURES

Случайные признаки
в каждом узле

$m \approx \sqrt{M}$ · классификация

Алгоритм построения леса

01

Бутстрап

Генерируем D_β с возвращением

02

Случайные признаки

В узле рассматриваем только m из M

03

Глубокое дерево

Растём до остановки; обычно без стрижки

04

Ансамбль

Голосование / среднее B деревьев

Каждое дерево может переобучиться — ансамбль снижает дисперсию

Ручной расчёт: 8 объектов

Индексы исходной выборки

0	1	2	3	4	5	6	7

БУТСТРАП — 8 ИЗВЛЕЧЕНИЙ

6	3	4	6	2	7	4	4

Объект 4 — 3 раза

Объект 6 — 2 раза

OOB: 0, 1, 5

$$P(OOB) = (1 - 1/N)^N \rightarrow 1/e \approx 0.368$$

OOB: бесплатная кросс-валидация

ДЛЯ КАЖДОГО ДЕРЕВА

Bootstrap обучение → **OOB-объекты валидация**

Объекты, не попавшие в конкретную bootstrap-выборку,
используются для оценки качества этого дерева.

≈ 63%

Уникальных объектов для обучения

≈ 37%

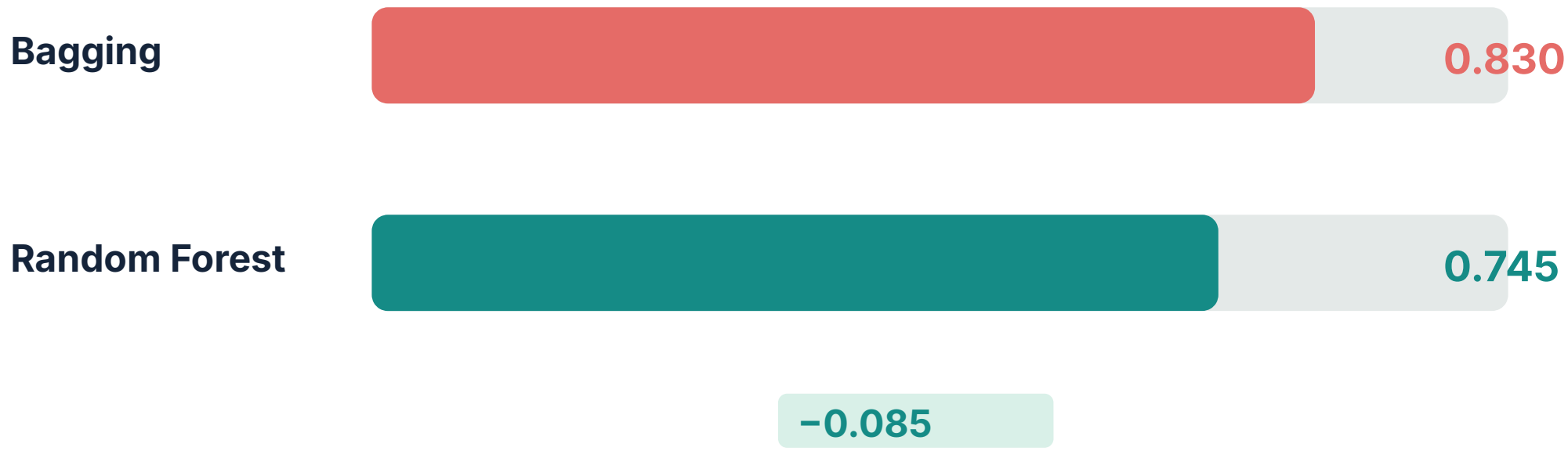
OOB для проверки

Быстро, без отдельной CV-схемы · возможна слегка заниженная оценка

ЭКСПЕРИМЕНТ

max_features снижает корреляцию

СРЕДНЯЯ КОРРЕЛЯЦИЯ ПРОГНОЗОВ ДЕРЕВЬЕВ



1000 объектов • 20 признаков • 5 информативных • по 50 деревьев

Bagging и Random Forest

```python
# 1. Чистый Bagging: каждое дерево видит ВСЕ 20 признаков
bag = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    max_features=1.0  # <--- Ключевое отличие
).fit(X, y)

# 2. Случайный лес: в каждом узле только sqrt(20) ≈ 4 признака
rf = RandomForestClassifier(
    n_estimators=50,
    max_features='sqrt' # <--- Случайность по признакам
).fit(X, y)

# Считаем среднюю попарную корреляцию предсказаний деревьев
print(f"Корреляция деревьев в Bagging: {get_corr(bag):.3f}")  # 0.830
print(f"Корреляция деревьев в RF:      {get_corr(rf):.3f}")   # 0.745
```

Гиперпараметры Random Forest

n_estimators	Число деревьев, обычно 100–500
max_depth	Максимальная глубина (None — без лимита)
min_samples_leaf	Минимальный размер листа
max_features	Случайные признаки в каждом узле

PYTHON · src/

```python
from sklearn.ensemble import
    RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=200,
    max_features='sqrt',
    max_depth=None,
    min_samples_leaf=1,
    oob_score=True,
    random_state=42
)
```

PYTHON

Практика: breast_cancer

0.9548

OOB-оценка

0.9708

Hold-out accuracy

569 объектов

30 признаков · 2 класса

200 деревьев · oob_score=True

PYTHON · src/

```
X, y = load_breast_cancer(
    return_X_y=True)
# train_test_split: 70% / 30%
rf.fit(X_train, y_train)
print(rf.oob_score_)
print(rf.score(X_test, y_test))
```

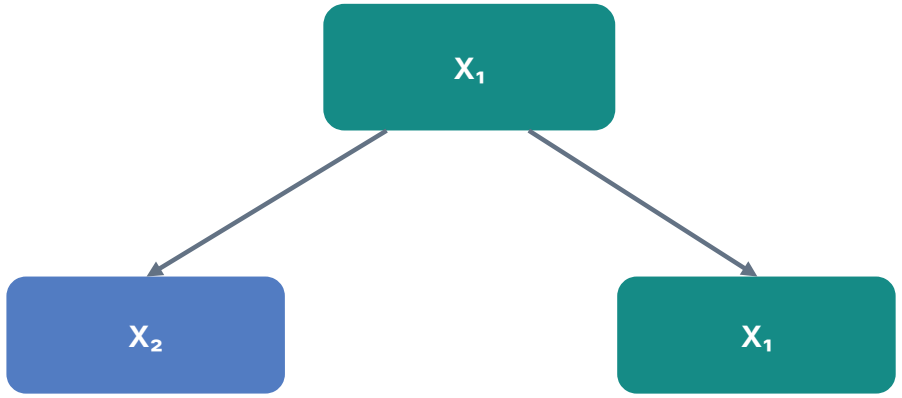
Интерпретация, ловушки и сравнения

02



Важность признаков • ограничения • альтернативы

Gini Importance: как считается



Выделяем все сплиты по X_1

MDI

Mean Decrease Impurity

Сумма уменьшений Gini
по всем узлам и деревьям

Вес ∝ числу объектов в узле

ЛОВУШКА

Смещение к высокой кардинальности

GINI IMPORTANCE · ПРИМЕР ИЗ ДОКЛАДА



Много уникальных значений → много случайных кандидатов на сплит

Решение: Permutation Importance

1 **Измерить** Качество на тесте

2 **Перемешать** Значения X_j

3 **Повторить** Оценку качества

4 **Сравнить** Падение метрики

0.510

Полезный признак · сильное падение accuracy

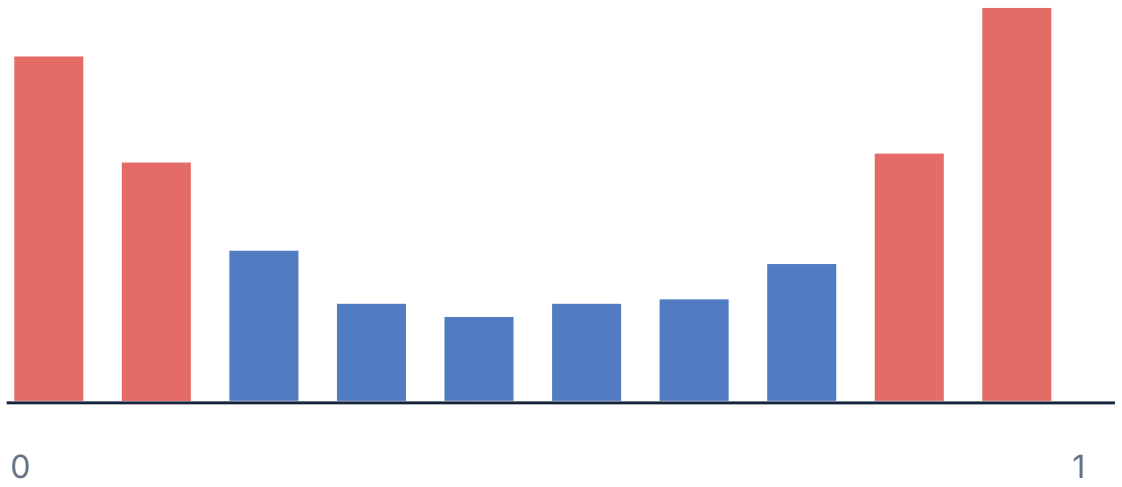
0.000

Шумовой признак · без падения

Алгоритм: sklearn.inspection.permutation_importance

Проблема: калибровка вероятностей

СХЕМА: РАСПРЕДЕЛЕНИЕ PREDICT_PROBA



Иллюстративная схема, не измеренная гистограмма

Частота голосов

≠

калиброванная вероятность

`CalibratedClassifierCV`

Сравнение с альтернативами

CART

Одинарное дерево

Проверяем: Дисперсия

Gradient Boosting

Последовательные деревья

Проверяем: Смещение

Линейная регрессия

Продолжает линейный тренд

Проверяем: Экстраполяция

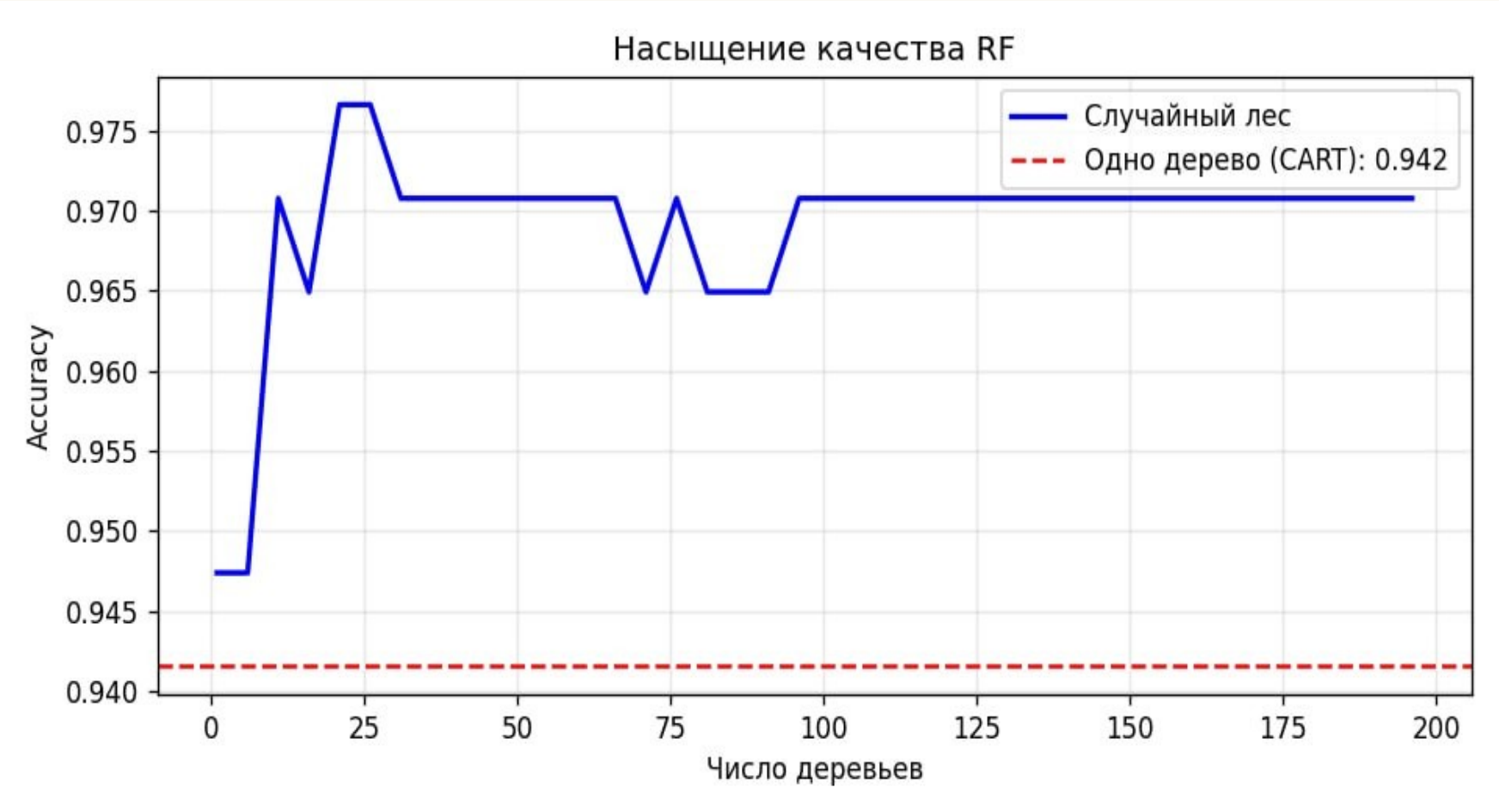
k-NN

Метод ближайших соседей

Проверяем: Масштаб и шум

СРАВНЕНИЕ

RF vs CART: насыщение качества



0.9415

CART · одно дерево

0.9708

RF · 200 деревьев

CART и Random Forest

```
# 1. Загружаем данные и делим на train/test
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

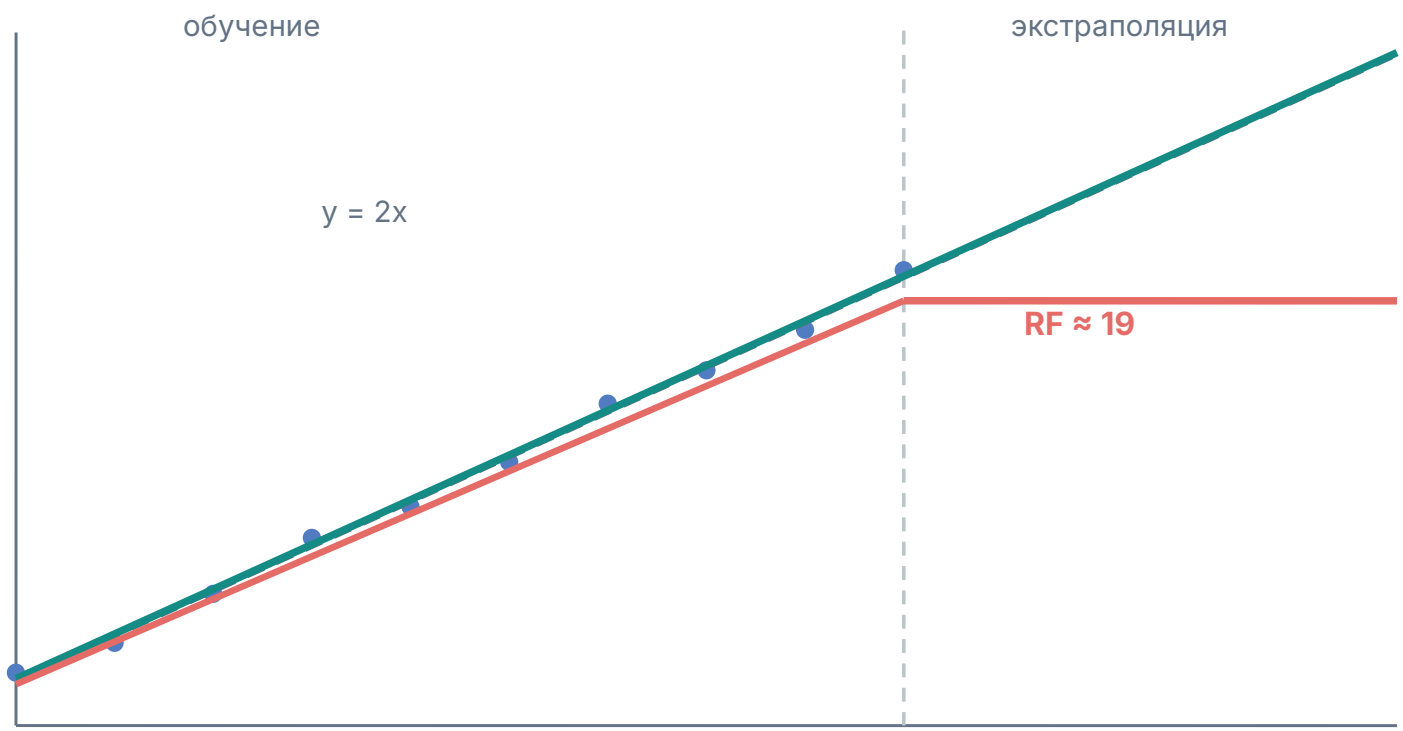
# 2. Обучаем одно "жадное" дерево CART (без ограничений глубины)
cart_model = DecisionTreeClassifier(random_state=42).fit(X_train, y_train)
cart_acc = cart_model.score(X_test, y_test) # Точность: ~0.941

# 3. Обучаем Случайный лес из 200 деревьев
rf_model = RandomForestClassifier(n_estimators=200, random_state=42).fit(X_train, y_train)

# 4. Считаем кумулятивную точность при добавлении деревьев (для графика)
rf_scores = []
for n in range(1, 201, 5):
    # Усредняем прогнозы первых n деревьев
    probs = np.mean([tree.predict_proba(X_test) for tree in rf_model.estimators_[:n]], axis=0)
    rf_scores.append((probs.argmax(axis=1) == y_test).mean())
```

СРАВНЕНИЕ

RF vs линейная регрессия: экстраполяция



0.3543

MAE · Linear Regression

6.6920

MAE · Random Forest

Экстраполяция

```python
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression

# 1. Данные: обучаемся на X от 1 до 10
X_train = np.linspace(1, 10, 50).reshape(-1, 1)
y_train = 2 * X_train.ravel()

# 2. Зона экстраполяции: просим предсказать X от 11 до 15
X_test = np.linspace(11, 15, 20).reshape(-1, 1)

# 3. Обучаем две модели на одних и тех же данных
rf = RandomForestRegressor(n_estimators=100, max_depth=5).fit(X_train, y_train)
lin = LinearRegression().fit(X_train, y_train)
```

СРАВНЕНИЕ

RF vs Gradient Boosting

RANDOM FOREST

ПАРАЛЛЕЛЬНО

T1 T2 T3

Независимые деревья
Снижение дисперсии
Хорошо «из коробки»

GRADIENT BOOSTING

ПОСЛЕДОВАТЕЛЬНО

T1 → T2 → T3

Деревья исправляют ошибки
Снижение смещения
Чувствителен к настройке

ФИНАЛЬНЫЙ ЭКСПЕРИМЕНТ

RF vs GB vs k-NN: «грязные» данные

50 шумовых признаков × 100 · БЕЗ StandardScaler

МЕТОД	ACCURACY	ВРЕМЯ FIT
k-NN (k=5)	0.924	0.001 с
Gradient Boosting	0.9591	0.5 с
Random Forest	**0.9708**	**0.131 с**

Время обучения из исходного выступления; зависит от оборудования.

Когда использовать Random Forest

ПРИМЕНЯТЬ

- Табличные данные
- Разнородные признаки
- Робастность к шуму
- Нет нужды экстраполировать
- Важна скорость обучения

НЕ ПРИМЕНЯТЬ

- Экстраполяция → линейная модель
- Максимум точности → бустинг
- Изображения/текст → нейросети
- Нужны правила → одно дерево

Пять главных выводов

01	**Два источника случайности**	Снижают дисперсию и корреляцию
02	**OOB — встроенная валидация**	Удобно, но оценка может быть чуть ниже
03	**Gini важность смещена**	Альтернатива: Permutation Importance
04	**Лес не экстраполирует**	Прогноз ограничен диапазоном обучения
05	**RF — надёжный baseline**	Для максимума качества сравните с бустингом

Источники

01

Leo Breiman

Random Forests / Machine Learning, 2001

02

Hastie · Tibshirani · Friedman

The Elements of Statistical Learning / Глава 15

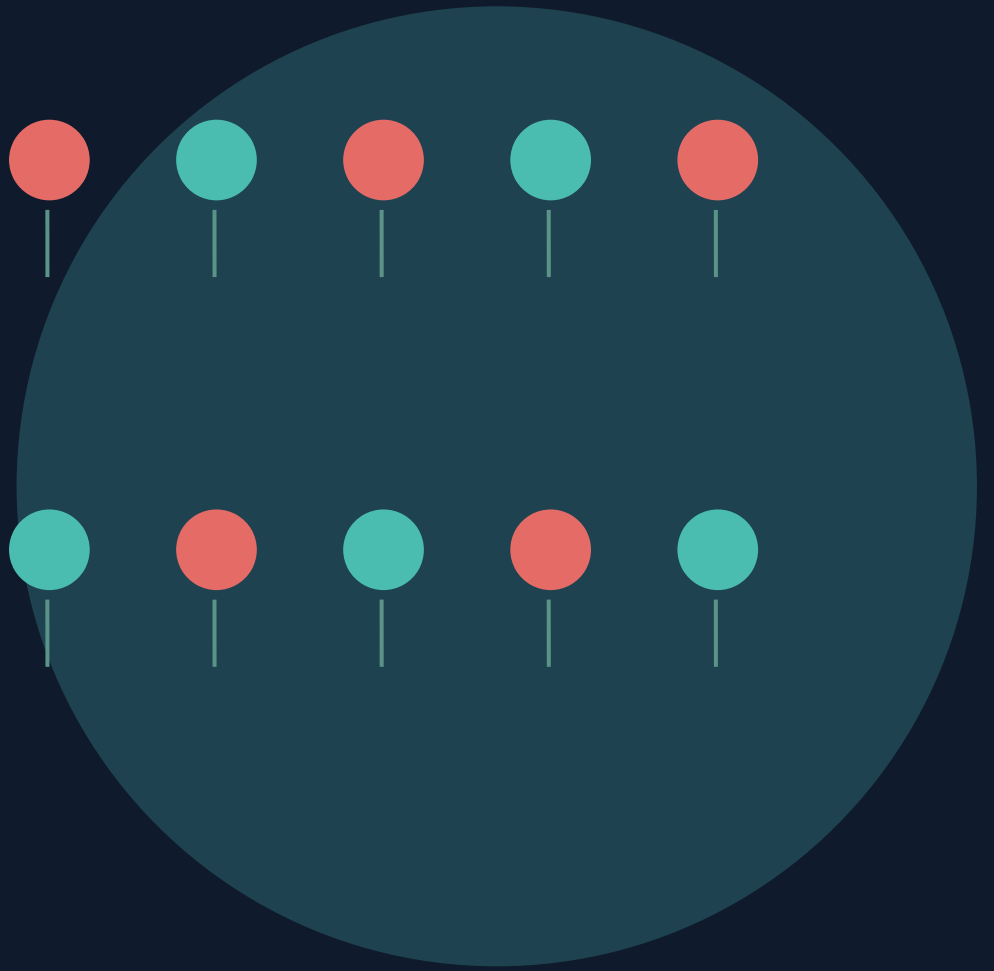
03

scikit-learn

Ensemble methods / Документация sklearn

Полные ссылки — в readme.md репозитория команды 17

Спасибо за внимание!



Команда 17